

MIDNIGHT HACKATHON

GAMING TRACK

CONFIDENTIAL BUILD SPEC v1.0

SHADOW PROTOCOL

Complete Midnight Integration Specification for the Python Game Server
APIs · Smart Contract · Bridge Service · Hooks · Tests · Build Plan

Base game: AI0702/Among-Us-clone (Python + pygame, Unlicense) — reskinned as *Shadow Protocol*

Chain: Midnight testnet · Compact smart contract · client-side ZK proofs

Build window: July 17–19, 2026 (~72 hours)

Audience: AI coding agents (Claude Code / Cursor) + human teammates. This document is the single source of truth.

Prime directive: Follow this spec exactly. Where the spec says VERIFY-AGAINST-DOCS, check the live Midnight documentation before writing code. Do not invent features beyond the Locked Scope (§3).

Table of Contents

1. How to Use This Document (Agent Operating Instructions)
2. System Overview, Pitch & Trust Model
3. Locked Scope & Non-Goals
4. Environment Setup (exact commands, versions, Docker, faucet)
5. Repository Layout (every file, every purpose)
6. Data Model: Seats, Roles, Commitments, Nullifiers, IDs
7. Smart Contract Specification — `shadow_ledger.compact`
8. Bridge Service Specification — `midnight-bridge/` (TypeScript)
9. HTTP API Reference — every endpoint, schema, status code, example
10. Python Integration — `midnight_hooks.py` + exact patch points in the base repo
11. Event Sequence Diagrams (deal, kill, vote, eject, game over, audit)
12. Audit Dashboard Specification
13. Error Handling, Queue & Retry Policy
14. Security & Privacy Checklist
15. Test Plan (contract, bridge, integration, regression) + how to run
16. 72-Hour Build Phases with Acceptance Gates
17. Demo Script & Submission Deliverables
18. Troubleshooting Appendix
19. Glossary

1. How to Use This Document (Agent Operating Instructions)

This specification is written to be executed by an AI coding agent inside VS Code with a human supervising. Rules of engagement:

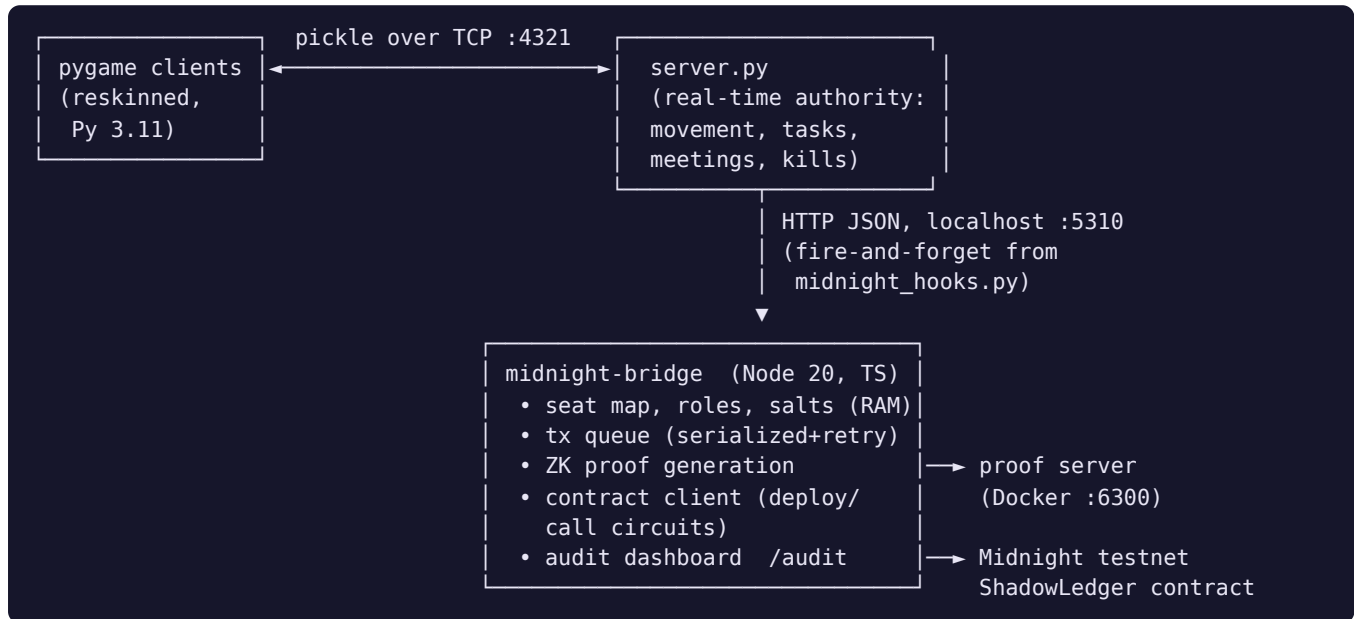
1. **Read this entire document before writing any code.** Then execute §16 (Build Phases) in order. Do not skip acceptance gates.
2. **Scope is locked.** If a feature is not in §3, do not build it, scaffold it, or leave TODOs for it. Deleting is allowed; adding is not.
3. **VERIFY-AGAINST-DOCS markers.** Compact and the Midnight JS SDK evolve quickly. Wherever this spec marks `VERIFY-AGAINST-DOCS`, fetch the current page (URLs given in §4.6) and pattern-match the *current* syntax/versions before writing that file. The *shapes* in this spec are binding; the exact identifiers may need updating.
4. **Chain calls never block gameplay.** Every call from Python to the bridge is asynchronous fire-and-forget (§10, §13). If you find yourself awaiting a proof inside the pygame loop, you have made an error.
5. **No secrets in logs.** Roles, salts, and seat → role mappings may only appear in: bridge memory, the contract's commitment openings at ejection/audit time, and the encrypted deal response. Never in stdout, error messages, socket broadcasts, or the dashboard before reveal time (§14).
6. **When behind schedule,** cut features strictly in the order of §16.8. Never cut items marked NEVER-CUT.
7. **Every file you create** must match the layout in §5. If you need a file not listed there, add it to §5's tree in the repo copy of this spec first (one-line justification as a comment).

2. System Overview, Pitch & Trust Model

2.1 One-paragraph pitch

The base repo is a working Among-Us-style LAN game — and its multiplayer dealer is literally rigged: **the impostor is the player with the highest `player_id`**. Shadow Protocol retrofits Midnight onto this game server so the role deal, every kill, every vote, and every ejection is bound to zero-knowledge commitments on the Midnight testnet. The server still runs the game in real time; it just can no longer lie about it. At game over, an on-chain audit reveal lets anyone verify the entire match. This is a **cryptographic fairness / anti-cheat layer for existing games**, demonstrated end-to-end on a real one.

2.2 Architecture



2.3 Division of responsibility

Component	Owns	Never does
server.py (Python)	Real-time game state, physics, tasks, meeting flow, bot AI; calls bridge hooks on the 5 canonical events	Never touches wallets, proofs, salts, or the contract; never blocks on the bridge
midnight-bridge (TS)	All Midnight interaction: seed, shuffle, commitments, wallets, proofs, tx queue, audit page	Never makes gameplay decisions; refuses game-inconsistent requests (belt-and-suspenders, §9)
ShadowLedger (Compact)	Binding record: commitments, kill log, vote nullifiers/tallies, ejections, winner, audit	Never stores a raw role or salt except through explicit reveal circuits
Audit dashboard	Judge-facing live event feed + endgame verification table	Never shows unrevealed roles

2.4 Trust model (put verbatim in README)

Threat	Base repo	Shadow Protocol v1	Mechanism
Rigged role assignment	Rigged by design (highest player_id)	Impossible	Deterministic Fisher–Yates from on-chain committed <code>gameSeed</code> ; audited at game end
Fake kill by a non-saboteur	Undetectable	Impossible	<code>recordKill</code> circuit requires ZK proof killer's committed role = SABOTEUR
Vote stuffing / double voting	Undetectable	Impossible	Per-meeting ZK nullifiers in contract state
Lying about an ejected player's role	Undetectable	Impossible	Ejection must open the day-one commitment
Rewriting match history	Trivial	Impossible	Append-only chain record + full audit reveal
Server/bridge reading secrets	Yes	Still yes — documented v1 limitation	Roadmap: browser clients holding own private state + Lace wallets (web edition)

Claim discipline: in the video and README, claim "the server can no longer forge or rewrite anything" — never claim "the server cannot see secrets." Precision here wins the track; overclaiming loses it in Q&A.

3. Locked Scope & Non-Goals

3.1 In scope (ALL of this, NOTHING else)

- 5 players per match (humans + base-repo bots). Roles: **1 SABOTEUR** (impostor), **4 AGENTS** (crew). Keep the repo's tasks and sabotage gameplay unchanged.
- Six on-chain event types: **deal, kill, vote, ejection+reveal, winner, audit reveal**.
- The bridge service, contract, Python hooks, and audit dashboard exactly as specified in §7–§12.
- Reskin: game renamed *Shadow Protocol*; sprite hue-shift; "Agent/Saboteur" wording; base-repo credit line.
- Tests in §15; deliverables in §17.

3.2 Explicit non-goals (do not build; mention only in README roadmap)

- Analyst/Medic roles, night-phase mechanics, hidden resources, trading, poker, fog-of-war.
- Mental-poker / MPC trustless dealing; threshold encryption; relayers.
- Player-held Lace wallets in the pygame client (bridge custodies seat keys — documented).
- Web game client; mobile; matchmaking; persistence/DB (bridge state is in-RAM per match; audit data also written to `audit_log.json`).
- Voice chat (`server_voice.py` , `voice.py` — leave untouched, excluded from run instructions).

4. Environment Setup

4.1 Host prerequisites

Tool	Version	Why / Notes
Python	3.11.x (hard requirement)	Base repo uses <code>asyncore</code> , removed in Python 3.12. Use a 3.11 venv. Fallback if impossible: <code>pip install pyasyncore</code> and add <code>import pyasyncore as asyncore</code> shim — decide in Phase 0 and record the decision in README.
Node.js	20 LTS+	Bridge service + Midnight JS SDK
Docker + compose	current	Runs the Midnight proof server
Compact toolchain	latest release	Official installer, then <code>compact update</code> ; verify with <code>compact compile --version</code> . VERIFY-AGAINST-DOCS.
Lace wallet (Midnight preview)	Chrome ext.	Only for the human deployer/faucet flow if needed; players do NOT need wallets

4.2 Python environment

```
# from repo root
python3.11 -m venv .venv
source .venv/bin/activate           # Windows: .venv\Scripts\activate
pip install -r requirements.txt     # create it: pygame, pytmx, requests (NEW dep for hooks)
python main.py                     # base game must run BEFORE any integration work
```

4.3 Compact toolchain + proof server

```
# Compact compiler — VERIFY-AGAINST-DOCS for the current installer URL and version
curl --proto 'https' --tlsv1.2 -LsSf \
  https://github.com/midnightntwrk/compact/releases/latest/download/compact-installer.sh | sh
compact update                      # pin the version it installs; write it into contract pragma
compact compile --version

# Proof server (leave running for the whole hackathon)
docker compose -f proof-server-local.yml up -d
curl -s http://127.0.0.1:6300 || echo "check container logs"
```

proof-server-local.yml (create at repo root — VERIFY-AGAINST-DOCS for current image tag; the example-bboard repo ships an equivalent file you can copy):

```
services:
  proof-server:
    image: midnightnetwork/proof-server:latest # pin the tag used by example-bboard
    ports: ["6300:6300"]
    command: ["midnight-proof-server", "--network", "testnet"]
    restart: unless-stopped
```

4.4 Bridge environment — `midnight-bridge/.env.example`

```
# --- network ---
MN_NETWORK_ID=TestNet                # VERIFY-AGAINST-DOCS: exact enum name in @midnight-
ntwrk/midnight-js-network-id
MN_INDEXER_URL=                      # public testnet indexer URL from docs quickstart
MN_INDEXER_WS_URL=                   # websocket variant
MN_NODE_URL=                         # public testnet node RPC from docs quickstart
MN_PROOF_SERVER_URL=http://127.0.0.1:6300
# --- wallet ---
MN_DEPLOYER_SEED=                    # 64-hex headless wallet seed; fund with tDUST from faucet;
NEVER commit real value
# --- contract ---
MN_CONTRACT_ADDRESS=                 # empty => bridge deploys fresh on boot and prints the
address
# --- bridge ---
BRIDGE_PORT=5310
BRIDGE_BIND=127.0.0.1                # localhost only; never expose
LOG_LEVEL=info
DEMO_FIXED_SEED=                     # hex; test-mode only, forces deterministic deal for B1 test
+ rehearsals
```

Phase 0, hour 1: create the deployer wallet, hit the testnet faucet for tDUST, and deploy the *stock example-counter* end-to-end. Faucet/latency problems are the #1 Midnight hackathon killer — surface them on Day 1, not Day 3.

4.5 Version pinning rule

Copy the exact `@midnight-ntwrk/*` dependency versions from the current `example-bboard` repo's `package.json`. Do not guess versions and do not mix versions across packages. Record the chosen set in README §"Versions".

4.6 VERIFY-AGAINST-DOCS reference list

Topic	URL
Compact language reference	docs.midnight.network/develop/reference/compact/lang-ref
Compact std-lib exports (persistentCommit/Hash, disclose)	docs.midnight.network/develop/reference/compact/compact-std-library/exports
Example dapp scaffold (structure, versions, compose file)	github.com/midnightntwrk/example-bboard · github.com/midnightntwrk/example-counter
Proof server usage	docs.midnight.network/develop/tutorial/using/proof-server
Counter tutorial (providers wiring)	docs.midnight.network/examples/counter
Base game repo	github.com/AI0702/Among-Us-clone

5. Repository Layout

```
shadow-protocol/      # fork/clone of AI0702/Among-Us-clone, renamed
├─ main.py            # base repo (entry point) – minor edits: title, hooks
init
├─ game.py            # base repo – patched at §10 patch points only
├─ server.py          # base repo – patched at §10 patch points only
├─ board.py / sprites.py / tasks.py # base repo – untouched except reskin
├─ menu.py / tilemap.py / gamefunctions.py # base repo – reskin only
├─ server_voice.py / voice.py        # base repo – UNTOUCHED, excluded from docs
├─ midnight_hooks.py                 # NEW – full listing in §10.2 (the ONLY new Python file)
├─ requirements.txt                   # NEW – pygame, pytmx, requests
├─ proof-server-local.yml             # NEW – §4.3
├─ midnight-bridge/                  # NEW – TypeScript workspace
│   └─ package.json / tsconfig.json / .env.example
│   └─ contract/
│       └─ src/shadow_ledger.compact # §7
│       └─ src/managed/              # compiler output: TS bindings + keys (gitignore large
artifacts)
│   └─ src/
│       └─ index.ts                  # boot: env, providers, deploy-or-join, start HTTP
│       └─ config.ts                 # env parsing + validation (zod)
│       └─ providers.ts              # indexer/node/proof/wallet providers wiring (§8.2)
│       └─ contractClient.ts         # typed wrappers over generated circuit calls (§8.3)
│       └─ game.ts                   # GameSession: seats, seed, shuffle, salts, role map
(§8.4)
│   └─ queue.ts                     # serialized tx queue + retries (§13)
│   └─ http.ts                      # Express app, routes (§9), validation, error format
│   └─ audit.ts                     # event log, verification, audit_log.json writer (§12)
│   └─ types.ts                     # all shared TS types (§8.5)
│   └─ public/audit.html            # dashboard single page (§12)
│   └─ test/                        # vitest: contract simulator + API tests (§15)
├─ DEMO_SCRIPT.md                  # §17
├─ README.md                       # §17.3 checklist
└─ SHADOW_PROTOCOL_SPEC.pdf        # this document
```

Reskin task list (Phase 0): window caption + menu title → "Shadow Protocol"; replace title-screen image; batch hue-shift sprite sheets (one-off Pillow script is fine, keep it in `tools/reskin.py`); replace strings "imposter/impostor" → "saboteur" and "crewmate" → "agent" in UI-facing text (keep internal variable names like `Minion.imposter` to minimize diff risk); add credit line "Engine derived from AI0702/Among-Us-clone (Unlicense)" to menu footer and README.

6. Data Model

6.1 Identifiers

Concept	Type	Definition
<code>player_id</code>	int	Base repo's random per-connection ID (1000–1000000), assigned by server.py. Bots also get one. Never sent on-chain.
<code>seat</code>	0–4	Canonical on-chain identity. Bridge assigns seats by ascending <code>player_id</code> order of the join list passed to /deal. The seat ↔ player_id map is public (dashboard shows it).
<code>role</code>	enum uint8	AGENT = 1 , SABOTEUR = 2 . (0 reserved = unknown.)
<code>matchId</code>	hex string	<code>hex(sha256(gameSeed contractAddress))[0:16]</code> — labels logs/dashboard; not on-chain.
<code>meetingRound</code>	uint	Starts 0; bridge increments after each ejection/meeting resolution (mirrors contract counter).

6.2 Randomness & the deal

1. Bridge generates `gameSeed = sha256(32 random bytes || bigint(now_ms))` — or `DEMO_FIXED_SEED` in test mode.
2. `gameSeed` goes on-chain in `initGame` **before** any role is communicated to the Python server. This ordering is the anti-rig guarantee: once the seed is committed, the shuffle is forced.
3. Shuffle: Fisher–Yates over `[SABOTEUR, AGENT, AGENT, AGENT, AGENT]` with a deterministic PRNG (xorshift or HMAC-DRBG) keyed by `gameSeed`. Same seed ⇒ same deal, always (test B1). `roles[seat] = shuffled[seat]`.

6.3 Commitments, salts, nullifiers

```
salt[seat]          = 32 random bytes (bridge CSPRNG, held in RAM + audit_log.json AFTER game over only)
roleCommit[seat]   = persistentCommit( [role[seat] as Field, seat as Field, gameSeed], salt[seat] )
voteNullifier      = persistentHash( [salt[voterSeat], meetingRound as Field, pad("vote")] )
```

- **Binding:** commitment includes `seat` and `gameSeed` so a commitment cannot be replayed for another seat or another match.
- **Nullifier:** derived from the secret salt + round + domain tag ⇒ one vote per seat per meeting, without linking votes to seats in the nullifier set. VERIFY-AGAINST-DOCS: exact `persistentCommit/persistentHash` signatures and argument encodings; keep the shapes above.
- **Custody (v1, documented):** the bridge holds all salts. Consequence: bridge can generate any seat's proofs. The chain still prevents *forgery inconsistent with commitments* (e.g., an AGENT-committed seat can never produce a valid kill). Roadmap: player-held salts in a web client.

7. Smart Contract Specification — `shadow_ledger.compact`

Syntax status: the listing below is **design pseudocode in Compact style**. Before writing the real file: (1) run the Compact installer and note your compiler version; (2) open the current language reference + std-lib exports + example-bboard contract; (3) transcribe this design into the *current* syntax (pragma, ledger ADT method names, witness declarations, `disclose()` placement, constructor form). Every check and effect below is binding.

7.1 Constants & state

```
pragma language_version >= X.Y;           // set to your installed compiler – VERIFY
import CompactStandardLibrary;

// ---- constants (encode as pure circuits or inline literals) ----
// PHASE_SETUP=0  PHASE_LIVE=1  PHASE_MEETING=2  PHASE_OVER=3
// ROLE_AGENT=1   ROLE_SABOTEUR=2
// WINNER_NONE=0  WINNER_AGENTS=1  WINNER_SABOTEUR=2
// SEATS=5

export ledger gamePhase: Uint<8>;
export ledger meetingRound: Counter;
export ledger gameSeed: Bytes<32>;
export ledger adminPk: Bytes<32>;           // deployer credential captured in constructor –
VERIFY mechanism (ownPublicKey)
export ledger roleCommitments: Map<Uint<8>, Bytes<32>>;
export ledger alive: Map<Uint<8>, Boolean>;
export ledger killCount: Counter;          // public body count
export ledger voteNullifiers: Set<Bytes<32>>;
export ledger voteTally: Map<Uint<8>, Uint<8>>;
export ledger ejectedRole: Map<Uint<8>, Uint<8>>; // seat -> revealed role
export ledger revealedRole: Map<Uint<8>, Uint<8>>; // endgame audit reveals
export ledger winner: Uint<8>;
```

7.2 Witnesses (private inputs supplied by the bridge's TS driver)

```
witness killerCreds(): [Uint<8>, Uint<8>, Bytes<32>]; // [role, seat, salt] of the killer
witness voterCreds(): [Uint<8>, Uint<8>, Bytes<32>]; // [role, seat, salt] of the voter
```

Witness values are attacker-controlled by definition — every circuit below re-derives the commitment from the witness and compares against ledger state. A witness that doesn't open the on-chain commitment must make the proof fail. Never trust a witness field without that check.

7.3 Circuit-by-circuit specification

7.3.1 `initGame(seed: Bytes<32>, c0..c4: Bytes<32>)` — admin, once

Specification	
Caller	Admin (deployer credential must match <code>adminPk</code>)
Preconditions	<code>gamePhase == SETUP</code> (this also enforces callable-once)
Effects	<code>gameSeed = seed</code> ; <code>roleCommitments[i] = c_i</code> for <code>i</code> in <code>0..4</code> ; <code>alive[i] = true</code> ; <code>gamePhase = LIVE</code>
Discloses	seed + commitments (public by design); zero role information

7.3.2 recordKill(victim: Uint<8>) — ZK, the showpiece

Specification	
Witness	[role, seat, salt] = killerCreds()
Checks (all must hold)	1. gamePhase == LIVE 2. role == ROLE_SABOTEUR 3. persistentCommit([role, seat, gameSeed], salt) == roleCommitments[seat] 4. alive[seat] == true (killer alive) 5. victim < 5 && alive[victim] == true && victim != seat
Effects	alive[disclose(victim)] = false; killCount.increment(1)
Discloses	Victim seat only. Killer seat is never disclosed — the chain records "a provably-legitimate saboteur killed seat N."

7.3.3 startMeeting() — admin

Requires gamePhase == LIVE. Sets gamePhase = MEETING; resets voteTally entries to 0.

7.3.4 recordVote(target: Uint<8>) — ZK

Specification	
Witness	[role, seat, salt] = voterCreds()
Checks	1. gamePhase == MEETING 2. persistentCommit([role, seat, gameSeed], salt) == roleCommitments[seat] 3. alive[seat] == true 4. target < 5 && alive[target] == true (skip-vote: target 255 allowed, tallied nowhere) 5. n = persistentHash([salt, meetingRound_value, "vote"]); !voteNullifiers.member(n)
Effects	voteNullifiers.insert(n); if target != 255: voteTally[disclose(target)] += 1
Discloses	Target only. Voter identity appears neither in the nullifier (salted hash) nor in effects. (Tx-sender metadata caveat documented §14.)

7.3.5 recordEjection(seat: Uint<8>, role: Uint<8>, salt: Bytes<32>) — admin, public reveal

Checks: gamePhase == MEETING; persistentCommit([role, seat, gameSeed], salt) == roleCommitments[seat]; alive[seat]. Effects: alive[seat]=false; ejectedRole[seat]=role; meetingRound.increment(1); gamePhase = LIVE. This is how the server is *forced* to tell the truth about an ejected player's role — a wrong (role, salt) cannot open the commitment.

7.3.6 endMeetingNoEjection() — admin

Checks phase MEETING; increments meetingRound; phase = LIVE. (Tie or skip-majority.)

7.3.7 declareWinner(w: Uint<8>) — admin, sanity-checked

Checks: phase != OVER; w ∈ {1,2}; consistency guards the contract *can* see: if any ejectedRole[s] == SABOTEUR then require w == WINNER_AGENTS. (Task-completion wins are attested by the server; the contract cannot see tasks — document this boundary.) Effects: winner = w; gamePhase = OVER.

7.3.8 auditReveal(seat: Uint<8>, role: Uint<8>, salt: Bytes<32>) — admin, endgame

Checks: gamePhase == OVER; commitment opens. Effects: revealedRole[seat] = role. Bridge calls this for all 5 seats at game over; dashboard then re-computes every commitment client-side and shows ✓/✗.

7.4 Contract invariants (each becomes a §15 test)

1. initGame callable exactly once; only by admin.
2. No circuit ever discloses a role except recordEjection / auditReveal (grep the compiled disclosures; test C10).
3. A seat whose commitment opens to AGENT can never produce a valid recordKill proof.

4. Vote nullifier prevents same-seat double vote within a meeting; permits voting again next meeting.
5. All action circuits are phase-guarded; dead seats can neither act nor be acted upon (except audit).
6. `recordEjection / auditReveal` with wrong role or salt must fail.

8. Bridge Service Specification — midnight-bridge/

8.1 package.json (shape — copy exact @midnight-ntwrk versions from example-bboard)

```
{
  "name": "midnight-bridge",
  "type": "module",
  "scripts": {
    "compact": "compact compile contract/src/shadow_ledger.compact contract/src/managed",
    "build": "npm run compact && tsc",
    "dev": "npm run build && node dist/index.js",
    "test": "vitest run"
  },
  "dependencies": {
    "@midnight-ntwrk/midnight-js-contracts": "match-example-bboard",
    "@midnight-ntwrk/midnight-js-indexer-public-data-provider": "match-example-bboard",
    "@midnight-ntwrk/midnight-js-node-zk-config-provider": "match-example-bboard",
    "@midnight-ntwrk/midnight-js-http-client-proof-provider": "match-example-bboard",
    "@midnight-ntwrk/midnight-js-network-id": "match-example-bboard",
    "@midnight-ntwrk/wallet": "match-example-bboard",
    "@midnight-ntwrk/wallet-api": "match-example-bboard",
    "express": "^4", "zod": "^3", "pino": "^9"
  },
  "devDependencies": { "typescript": "^5", "vitest": "^2", "@types/express": "^4" }
}
```

8.2 providers.ts

Wire the standard provider set exactly as the current example-counter/bboard does (VERIFY-AGAINST-DOCS): public data provider → `MN_INDEXER_URL/WS`; zk config provider → path to `contract/src/managed/shadow_ledger` keys; proof provider → `MN_PROOF_SERVER_URL`; wallet + midnight providers from headless wallet built from `MN_DEPLOYER_SEED`; network id set globally from `MN_NETWORK_ID`. Export a single `buildProviders(): Promise<Providers>`. On boot, log wallet address + tDUST balance; **refuse to start if balance is 0** with message "fund via faucet".

8.3 contractClient.ts

Thin typed wrapper over the generated bindings: `deploy()`, `join(address)`, and one method per circuit (§7.3) returning `{ txId: string; blockHeight?: number }`. Witness implementations for `killerCreds` / `voterCreds` read from a per-call context object set by `GameSession` (the generated API passes witnesses at contract construction — follow the current example's witness wiring pattern; VERIFY-AGAINST-DOCS). No other file touches the generated code.

8.4 game.ts — GameSession

```
class GameSession {
  state: 'IDLE' | 'LIVE' | 'MEETING' | 'OVER';
  seatByPlayerId: Map<number, number>; playerIdBySeat: number[];
  roles: Role[]; // SECRET – never serialized except audit after OVER
  salts: Uint8Array[]; // SECRET – same rule
  gameSeed: Uint8Array; meetingRound: number; aliveSeats: Set<number>;

  async deal(playerIds: number[]): Promise<DealResult>
  // 1. require state==IDLE and playerIds.length==5 (else 409/422)
  // 2. seats = playerIds sorted ascending, index = seat
  // 3. gameSeed = DEMO_FIXED_SEED ?? sha256(randomBytes(32) || now)
  // 4. roles = fisherYates([SAB,AG,AG,AG,AG], drbg(gameSeed))
  // 5. salts = 5 x randomBytes(32); commits per §6.3
  // 6. await contract.initGame(seed, commits) // ON-CHAIN FIRST – anti-rig ordering
  // 7. state=LIVE; return { seatMap, saboteurPlayerId, commitments, txId }
  kill(killerId, victimId) // validate belt-and-suspenders (§9 /kill) then queue recordKill
  vote(voterId, targetId) // queue recordVote (auto startMeeting on first vote of a round)
  eject(seatOrNone) // queue recordEjection or endMeetingNoEjection
  gameover(winner) // queue declareWinner + auditReveal x5; write audit_log.json;
  state=OVER
}
```

8.5 types.ts — canonical shared types

```
export type Role = 1 | 2; // 1 AGENT, 2 SABOTEUR
export type Winner = 1 | 2; // 1 agents, 2 saboteur
export interface DealRequest { playerIds: number[] } // exactly 5, unique, ints > 0
export interface DealResult { matchId: string; contractAddress: string;
  seatMap: { playerId: number; seat: number }[];
  saboteurPlayerId: number; // ONLY field the Python server needs secretly
  commitments: string[]; gameSeedHex: string; txId: string }
export interface KillRequest { killerId: number; victimId: number }
export interface VoteRequest { voterId: number; targetId: number | null } // null = skip
export interface EjectRequest { round: number; ejectedId: number | null } // null = tie/skip
export interface GameOverRequest { winner: Winner; reason: 'tasks' | 'ejection' | 'kills' | 'sabotage' }
export interface BridgeError { error: string; code: ErrorCode; detail?: string }
export type ErrorCode = 'BAD_REQUEST' | 'WRONG_STATE' | 'UNKNOWN_PLAYER' | 'NOT_SABOTEUR'
  | 'DEAD_ACTOR' | 'DEAD_TARGET' | 'DUPLICATE' | 'CHAIN_UNAVAILABLE' | 'INTERNAL';
export interface AuditEvent { seqNo: number; kind: 'deal' | 'kill' | 'vote' | 'eject' | 'no_eject'
  | 'winner' | 'reveal'; publicData: Record<string, unknown>;
  status: 'queued' | 'proving' | 'submitted' | 'confirmed' | 'failed'; txId?: string; ts: string }
```

9. HTTP API Reference

Base URL `http://127.0.0.1:5310`. All bodies JSON (Content-Type: `application/json`). All errors use the `BridgeError` shape with appropriate HTTP status. The API is **localhost-only by binding**; there is no auth in v1 (documented). Every mutating route is validated with zod against §8.5 request types → `400 BAD_REQUEST` with field detail on failure.

GET /health

Liveness + dependency check. Use as the Phase 0/3 gate and in DEMO_SCRIPT preflight.

```
200 { "ok": true, "network": "TestNet", "contractAddress": "...", "walletAddress": "...",
      "balanceTDust": "...", "proofServer": "up", "indexer": "up", "queueDepth": 0,
      "session": "IDLE|LIVE|MEETING|OVER" }
503 { "ok": false, "code": "CHAIN_UNAVAILABLE", "detail": "proof server unreachable at ..." }
```

POST /deal

Synchronous (the one route that waits for chain confirmation — the game must not start unanchored). Timeout guidance for caller: 120 s.

```
Request { "playerIds": [4213, 88231, 90111, 152003, 731188] }
201 → DealResult (§8.5) # saboteurPlayerId: Python server keeps secret, sets Minion.imposter
409 WRONG_STATE - session not IDLE (send POST /reset first in dev)
422 BAD_REQUEST - not exactly 5 unique positive ints
503 CHAIN_UNAVAILABLE - deal MUST fail loudly; Python aborts match start (§10.3-P1)
```

POST /kill

Async: returns immediately after validation + enqueue; proof/tx happen in the queue worker.

```
Request { "killerId": 152003, "victimId": 4213 }
202 { "accepted": true, "seqNo": 7 }
404 UNKNOWN_PLAYER - id not in seat map
409 WRONG_STATE - session not LIVE
409 NOT_SABOTEUR - killerId is not the dealt saboteur (belt; contract is suspenders)
409 DEAD_ACTOR / DEAD_TARGET
422 BAD_REQUEST - killer == victim
```

POST /vote

```
Request { "voterId": 88231, "targetId": 90111 } # targetId null = skip vote
202 { "accepted": true, "seqNo": 12 } # first vote of a round auto-queues
startMeeting
409 DUPLICATE - voter already voted this round (bridge-side pre-check mirrors nullifier)
404 / 409 / 422 - as /kill semantics
```

POST /eject

```
Request { "round": 0, "ejectedId": 90111 }           # ejectedId null = tie/skip →
endMeetingNoEjection
202 { "accepted": true, "seqNo": 15, "willReveal": true }
409 WRONG_STATE – not in MEETING; 404 UNKNOWN_PLAYER; 409 DEAD_TARGET
422 BAD_REQUEST – round != current meetingRound (stale duplicate protection)
```

POST /gameover

```
Request { "winner": 1, "reason": "ejection" }       # winner: 1 agents, 2 saboteur
202 { "accepted": true, "auditReveals": 5 }         # queues declareWinner + 5 × auditReveal
409 WRONG_STATE – already OVER (idempotent-ish: second call returns 409, not a double submit)
```

POST /reset

Dev/demo only: clears the in-RAM session (does NOT touch the chain; a new /deal deploys or reuses per config). Guard behind `NODE_ENV !== 'production'`.

```
200 { "ok": true }
```

GET /events

Server-Sent Events stream of `AuditEvent` JSON (one per `data:` line). The dashboard subscribes here. Replays the full event log on connect, then streams updates (status transitions re-emit the event with the same `seqNo`).

GET /audit and GET /audit/data

/audit serves `public/audit.html`. /audit/data returns the verification payload:

```
200 { "matchId": "...", "contractAddress": "...", "gameSeedHex": "...",
      "seatMap": [...], "events": [AuditEvent...],
      "reveals": [ { "seat": 0, "role": 1, "saltHex": "...", "commitmentHex": "...",
                    "recomputedHex": "...", "match": true } ... ] # reveals: [] until OVER }
```

9.1 Global API rules

- **Ordering:** the queue is strictly FIFO by `seqNo`; the bridge never reorders game events.
- **Idempotency:** Python hooks may retry on connection errors; the bridge dedupes: identical /kill for an already-dead victim → `409 DEAD_TARGET`; /vote duplicate → `409 DUPLICATE`; /eject stale round → `422`. Hooks treat 4xx as "already handled / drop", 5xx as "retry later".
- **Secrecy:** no response except /deal (and /audit/data after OVER) ever contains a role or salt. `saboteurPlayerId` appears ONLY in the /deal response.

10. Python Integration

10.1 Design rules

- One new file: `midnight_hooks.py`. Patches to `server.py / game.py` are minimal and marked with `# [MIDNIGHT]` comments so the diff is reviewable.
- All hook calls are **non-blocking**: a single worker thread drains a `queue.Queue`; HTTP timeout 5 s; failures log a console warning and never raise into the game loop.
- Exception: `deal()` is **blocking by design** (match must not start unanchored) — called during lobby/loading, not during the frame loop.

10.2 Full listing — midnight_hooks.py

```
# [MIDNIGHT] Bridge client. Python 3.11. Dependency: requests.
import json, queue, threading, logging
import requests

log = logging.getLogger("midnight")
BRIDGE = "http://127.0.0.1:5310"

class MidnightHooks:
    """Fire-and-forget client for the midnight-bridge. Never raises into gameplay."""

    def __init__(self, base_url: str = BRIDGE):
        self.base = base_url
        self.q: "queue.Queue[tuple[str, dict]]" = queue.Queue()
        self.round = 0
        self.enabled = True
        t = threading.Thread(target=self._worker, daemon=True, name="midnight-hooks")
        t.start()

    # ----- blocking, lobby-time only -----
    def deal(self, player_ids: list[int]) -> dict:
        """Anchors the match. RAISES on failure – caller must abort match start.
        Returns DealResult; caller uses ONLY seatMap + saboteurPlayerId."""
        r = requests.post(f"{self.base}/deal",
                          json={"playerIds": sorted(player_ids)}, timeout=120)
        r.raise_for_status()
        d = r.json()
        log.info("match anchored: matchId=%s contract=%s tx=%s",
                 d["matchId"], d["contractAddress"], d["txId"])
        return d # NOTE: never log d["saboteurPlayerId"]

    # ----- non-blocking gameplay events -----
    def kill(self, killer_id: int, victim_id: int) -> None:
        self._enqueue("/kill", {"killerId": killer_id, "victimId": victim_id})

    def vote(self, voter_id: int, target_id: int | None) -> None:
        self._enqueue("/vote", {"voterId": voter_id, "targetId": target_id})

    def eject(self, ejected_id: int | None) -> None:
        self._enqueue("/eject", {"round": self.round, "ejectedId": ejected_id})
        self.round += 1

    def gameover(self, winner: int, reason: str) -> None:
        self._enqueue("/gameover", {"winner": winner, "reason": reason})

    # ----- internals -----
    def _enqueue(self, path: str, body: dict) -> None:
        if self.enabled:
            self.q.put((path, body))

    def _worker(self) -> None:
        while True:
            path, body = self.q.get()
            for attempt in range(30): # ~2.5 min of retries
                try:
                    r = requests.post(self.base + path, json=body, timeout=5)
                    if r.status_code < 500:
                        if r.status_code >= 400:
                            log.warning("bridge rejected %s: %s %s",
                                         path, r.status_code, r.text[:200])
                            break # 2xx ok; 4xx = drop (handled/dup)
                except requests.RequestException as e:
                    log.warning("bridge unreachable (%s), retry %d", e.__class__.__name__,
                                attempt)
```

```

attempt)
    threading.Event().wait(min(2 ** attempt, 15))    # capped backoff
    self.q.task_done()

hooks = MidnightHooks()    # module singleton: `from midnight_hooks import hooks`

```

10.3 Patch points in the base repo

Function/variable names below were verified by reading the repo (`server.py`: `asyncore` dispatcher on :4321, `minionmap`, `Minion.imposter`, `victim_id`, `voted/got_votes`; `game.py`: `runmultiplayer()`, kill via `K_RETURN` + `victim_id`, ejection at `got_votes >= 2`, win checks incl. `missions_done == 8`). Exact line numbers will differ — locate by the anchors given; each patch is a few lines.

#	Anchor (find this logic)	Patch
P1 Deal	Multiplayer role assignment: the branch that sets <code>imposter = True</code> for the highest <code>player_id</code> (server-side, when the lobby is full / game starts)	DELETE the highest-id rule entirely. Replace with: <pre> d = hooks.deal([m.player_id for m in minionmap.values()]) sab = d["saboteurPlayerId"] for m in minionmap.values(): m.imposter = (m.player_id == sab) </pre> Wrap in try/except → on failure show "Fairness layer unreachable — match aborted" and return to lobby. No silent fallback.
P2 Kill	Server-side processing of an incoming <code>victim_id</code> (the world-update path in <code>updateWorld()</code> that marks a player dead)	After the existing death handling: <pre> hooks.kill(killer.player_id, victim.player_id) </pre> (the killer is the Minion whose payload carried <code>victim_id</code>)
P3 Vote	Server-side processing where a player's <code>voted</code> choice increments the target's <code>got_votes</code>	<code>hooks.vote(voter.player_id, target.player_id)</code> — and for a skip: <code>hooks.vote(voter.player_id, None)</code> . Guard so each player's vote posts once per meeting (mirror the flag the repo uses to stop double counting).
P4 Eject	Ejection trigger: <code>got_votes >= 2</code> → <code>alive_status = False</code> + eject screen	<code>hooks.eject(ejected.player_id)</code> ; in the tie/no-ejection path: <code>hooks.eject(None)</code> . Exactly one call per meeting.
P5 Game over	Each multiplayer win branch: all tasks done (<code>missions_done == 8</code> / per-player task loop), saboteur ejected, all agents dead, sabotage timer expiry	<code>hooks.gameover(1, "tasks")</code> · <code>hooks.gameover(1, "ejection")</code> · <code>hooks.gameover(2, "kills")</code> · <code>hooks.gameover(2, "sabotage")</code> respectively. Exactly one call per match.
P6 HUD (optional polish)	pygame HUD draw	Poll nothing — just render a static "☼ Shadow Protocol: match anchored <matchId>" line from the deal result. (Live per-tx toasts are cut-list material.)

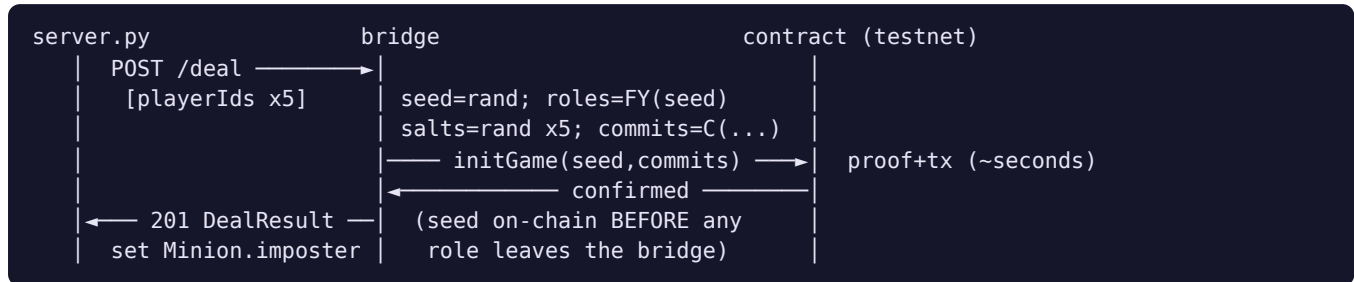
P1 is the heart of the demo. Keep a screenshot of the original highest-player_id line for the video's "before" shot, then delete it. Test R2 greps that no code path can reach the old rule.

10.4 Bots

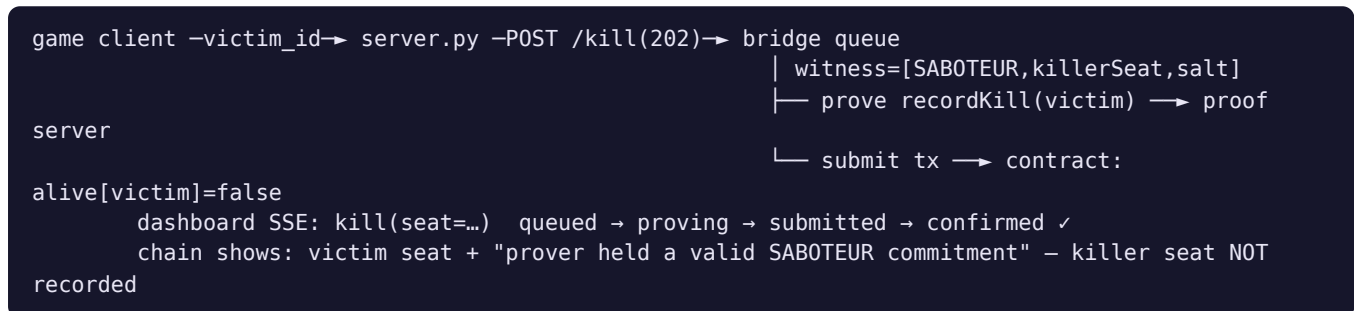
Base-repo bots keep working unchanged: their kills/votes flow through the same server-side paths as humans, so P2–P4 fire for them automatically. If a bot code path bypasses `victim_id` (freeplay-style direct `bot.alive_status=False`), route multiplayer bot kills through the same handler so the hook fires — single-player freeplay mode stays un-hooked and out of demo scope.

11. Event Sequence Diagrams

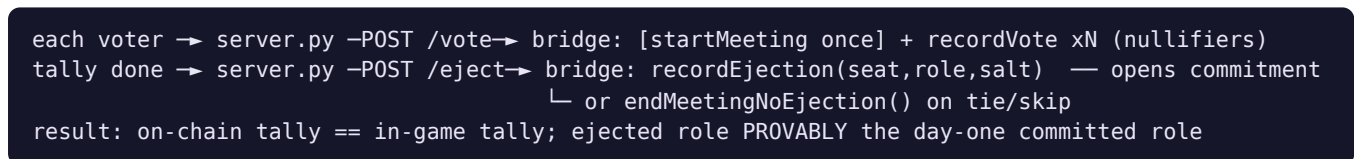
11.1 Deal (synchronous, anti-rig ordering)



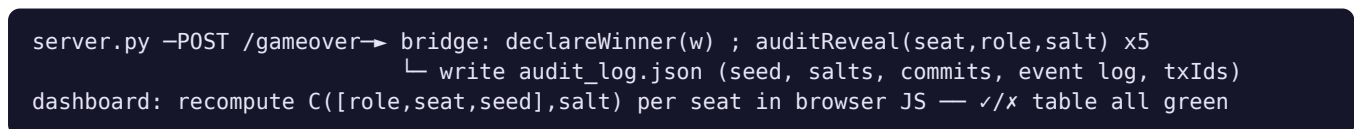
11.2 Kill



11.3 Vote → Ejection



11.4 Game over → Audit



12. Audit Dashboard Specification ([public/audit.html](#))

Single self-contained HTML file (inline CSS/JS, dark theme, no build step). Sections top-to-bottom:

1. **Header:** "Shadow Protocol — Match Audit" · matchId · contract address (monospace, copy button) · network badge · session state.
2. **Fairness banner:** one sentence: "Roles were committed on-chain at *deal time*; every kill and vote below carries a zero-knowledge proof; nothing can be rewritten."
3. **Live event feed** (SSE from `/events`): table — seq · time · kind · public data (e.g. `kill → seat 3`, `vote → seat 2 (+1)`) · status chip `queued` / `proving` / `submitted` / `confirmed` / `failed` · txId short-link. **Never display roles, salts, killer seats, or voter seats.**
4. **Commitment table:** seat · playerId · commitment hex (short) · status ("hidden " until reveal).
5. **Endgame verification** (appears when `/audit/data` has reveals): per seat — revealed role · salt · stored commitment · *browser-recomputed* commitment · ✓/X. Verification MUST run client-side in the page's JS (import or inline the same commitment routine the bridge uses, compiled for the browser, or re-derive via the generated

binding bundle — VERIFY feasibility; fallback: bridge sends `recomputedHex` AND the page hash-checks equality and labels it "bridge-recomputed"). Big green "ALL COMMITMENTS VERIFIED" banner when 5/5 match.

6. **Trust model footnote:** the §2.4 table, condensed.

This page is a submission deliverable, not a debug tool. It is what judges look at while the pygame demo runs. Budget real polish time (Phase 4).

13. Error Handling, Queue & Retry Policy

13.1 Bridge tx queue (`queue.ts`)

- Single worker, strict FIFO by `seqNo` . One in-flight tx at a time (avoids nonce/ordering races). Each job: build witness context → prove → submit → await confirmation → emit status via SSE at every transition.
- **Retries:** transient failures (proof server 5xx/conn, node timeouts) → exponential backoff 1 s → 2 → 4 → ... → cap 30 s, max 10 attempts, then mark `failed` (red in dashboard) and CONTINUE with the next job — one poisoned tx must not stall the match record.
- **Non-retryable:** proof *verification* rejection or circuit assertion failure = logic bug or forgery attempt → mark failed immediately, log at error, never retry.
- Queue depth surfaces in `/health` ; if depth > 20 log a warning (proofs too slow for game pace — expected worst case ~1 event/5 s in this game, fine).

13.2 Failure matrix

Failure	During	Behavior
Bridge down	/deal	Match start ABORTS with visible error (no unanchored games)
Bridge down	gameplay events	Python hook queue retries ~2.5 min with backoff; game unaffected; events land late but in order
Proof server down	any queued tx	Bridge retries per 13.1; dashboard shows 🕒 ; <code>/health</code> → 503
Testnet congestion	any	Same as above; confirmations lag, order preserved
Circuit assertion fails	recordKill etc.	Non-retryable; ✖ in dashboard. In an honest match this indicates a bridge/game desync bug — investigate immediately (test B4 exercises this path deliberately)
Bridge crash mid-match	—	v1 accepts loss of pending events (RAM state); <code>audit_log.json</code> has everything up to last write; documented limitation

14. Security & Privacy Checklist (run before submission)

#	Check	How
S1	No role/salt in any log	<code>grep -riE "saboteur salt role" logs/ *.log</code> + review pino output at info level; roles/salts only in audit_log.json (written at OVER)
S2	<code>saboteurPlayerId</code> never broadcast	It exists only in the /deal HTTP response and server.py memory; grep pickle payload construction to confirm the imposter flag is only sent to the owning client (base repo behavior — verify, since pickle syncs many fields; if the repo leaks <code>imposter</code> to all clients, that's a base-game flaw: note it in README, out of scope to fix)
S3	Bridge binds 127.0.0.1 only	<code>ss -tlnp grep 5310</code>
S4	No secrets in repo	<code>.env</code> gitignored; <code>MN_DEPLOYER_SEED</code> absent from history; audit_log.json gitignored
S5	Killer/voter seats absent from chain + dashboard	Inspect confirmed tx public data + /audit/data before OVER
S6	Old dealer rule unreachable	Test R2 grep + code review of P1
S7	Pickle caveat documented	README: pickle over LAN is inherited from the base repo, trusted-LAN demo only
S8	Claim discipline	README/video say "cannot forge or rewrite", never "cannot see"; tx-sender metadata note included (all txs come from the bridge wallet, which is itself a privacy feature here: individual players are not linkable on-chain)

15. Test Plan

15.1 How to run

```
# contract + bridge (vittest; contract tests use the compiled contract's simulator/test harness
# per the current example-bboard testing pattern – VERIFY-AGAINST-DOCS)
cd midnight-bridge && npm test

# python integration (pytest, mocks bridge with a local http server fixture)
source .venv/bin/activate && pytest tests/ -v

# full-stack bot game (Phase 3 gate): bridge up + proof server up + testnet
python main.py --demo-bots          # or the repo's multiplayer path with 5 bot clients
```

15.2 Contract tests (vittest + simulator) — MUST ALL PASS before Phase 2 ends

#	Case	Expected
C1	<code>initGame</code> called twice / by non-admin	Both rejected
C2	<code>recordKill</code> witness role=AGENT (commitment opens correctly for AGENT)	Proof/assertion fails
C3	Valid saboteur kill	victim dead, killCount+1; killer seat absent from public state
C4	Kill: dead victim / dead killer / victim==killer / phase != LIVE	All rejected
C5	Same voter votes twice in one meeting (same salt+round)	Second rejected (nullifier)
C6	Same voter votes in next meeting (round incremented)	Accepted
C7	<code>recordVote</code> witness salt not matching commitment	Proof fails
C8	<code>recordEjection</code> wrong role or wrong salt	Rejected
C9	Saboteur ejected then <code>declareWinner(2)</code>	Rejected; <code>declareWinner(1)</code> accepted
C10	Disclosure audit: compile output + circuit review	No role disclosure outside ejection/audit circuits
C11	<code>auditReveal</code> before OVER / with bad opening	Rejected
C12	Vote target = dead seat; vote target = 255 (skip)	Rejected; accepted with no tally

15.3 Bridge tests (vitest, HTTP level)

#	Case	Expected
B1	Two /deal runs with same DEMO_FIXED_SEED	Identical seatMap + saboteur seat (determinism)
B2	20 rapid valid events	All 202; queue drains FIFO; seqNos strictly increasing; no reorder
B3	Proof server stopped, then restarted mid-queue	Events retry, then confirm; none lost; /health reflected 503 → 200
B4	/kill with a non-saboteur killerId	409 NOT_SABOTEUR (belt); direct contract call with agent witness also fails (suspenders, = C2)
B5	/vote duplicate voter same round	409 DUPLICATE; nothing enqueued twice
B6	/deal with 4 ids, 6 ids, duplicate ids	422 with field detail
B7	Wrong-state calls: /kill when IDLE, /gameover twice	409 WRONG_STATE
B8	/audit/data before OVER	reveals: [], no role/salt anywhere in payload (assert by regex)

15.4 Python + full-stack tests

#	Case	Expected
P1	Bridge down at match start	Match aborts with visible error; no game without anchor
P2	Bridge down during play (kill fired)	Game unaffected; hook retries; event lands after bridge returns
P3	Frame-time with hooks active vs disabled	No measurable frame-time change (hooks are queue-put only)
P4	Full 5-bot match, start → finish	All 6 event kinds on-chain; dashboard verification 5/5 ✓ green
R1	Py 3.11 venv: server + 2 LAN clients	Base game works (asyncore alive)
R2	grep -rn "highest max(.*player_id" server.py game.py + review	Old dealer rule gone from all reachable paths
R3	Ejection at got_votes >= 2 with 5 players; tie case	On-chain tally matches in-game; tie → endMeetingNoEjection
R4	Reskin grep: "Among Us", Innersloth asset filenames in UI	Only the credit line remains

16. 72-Hour Build Phases with Acceptance Gates

Phase	When	Work	GATE (all must pass)
0 · Foundations	Jul 17 AM (≤4 h)	Py 3.11 venv; base game runs (LAN, 2 clients); compact installed; proof server up; deployer wallet + faucet tDUST; deploy stock example-counter to testnet; scaffold §5 tree; reskin pass	R1 ✓ · stock example round-trips on testnet · <code>compact compile --version</code> pinned into pragma
1 · Contract	Jul 17 PM	Write <code>shadow_ledger.compact</code> per §7 (current syntax); compile; simulator tests	C1–C12 green
2 · Bridge	Jul 18 AM	providers, contractClient, GameSession, queue, HTTP API per §8–§9; deploy to testnet	B1–B8 green · <code>/health 200</code> against live testnet
3 · Hooks	Jul 18 PM	midnight_hooks.py + patches P1–P5; bot routing (§10.4)	P1–P4, R2–R3 green · full bot match writes all 6 event kinds on-chain
4 · Dashboard	Jul 18 eve	audit.html per §12, SSE feed, endgame verification, polish	P4 re-run: dashboard 5/5 ✓ green live during a match
5 · Hardening	Jul 19 AM	§14 checklist S1–S8; 3× full E2E rehearsal; record backup screen-capture of a complete run NOW	All S-checks pass · backup video exists
6 · Ship	Jul 19 PM (protect 4 h)	README (§17.3), DEMO_SCRIPT.md, ≤2-min video, repo public, submission form	Clean-machine quickstart tested · video uploaded · submitted

16.8 Cut order if behind (strict)

- Vote ZK circuit → votes become admin-recorded events (keep nullifier out; lose test C5–C7; keep the kill circuit).
- Browser-side commitment recomputation → bridge-recomputed with page-side equality check.
- P6 HUD line; sprite hue-shift beyond the title screen (name change stays).
- `/events SSE` → dashboard polls `/audit/data` every 2 s.

NEVER-CUT: deal commitments + anti-rig ordering (P1) · `recordKill` role proof · ejection commitment opening · audit reveal + dashboard table · README trust model + base-repo disclosure · the video.

17. Demo Script & Submission Deliverables

17.1 Live/recorded demo layout

Screen split: pygame window left, audit dashboard right. Preflight: `GET /health 200`, proof server up, one rehearsed 5-bot-plus-1-human match path using `DEMO_FIXED_SEED`.

17.2 ≤2-minute video beats

Time	Beat
0:00–0:12	"This is Shadow Protocol at [event name]." Show the base repo's original line: <code>impostor = highest player_id</code> . "Every centralized game has this — you just can't usually read the source."
0:12–0:35	Match start: dashboard shows <code>initGame</code> tx — seed + 5 commitments on-chain <i>before</i> roles exist in the game. "The deal is now forced. Nobody — including our server — can pick the saboteur."
0:35–1:00	A kill in-game → dashboard event goes proving → confirmed. "The chain just verified a zero-knowledge proof that the killer's committed role is Saboteur — without recording who the killer is."
1:00–1:20	Meeting: votes land with nullifiers (show a rejected duplicate from the test suite, one screenshot); ejection opens the day-one commitment — role reveal is provable, not asserted.
1:20–1:45	Game over → audit table flips to 5/5 ✓ green. "Anyone can re-verify this match forever."
1:45–2:00	Trust table one-liner + "built on Midnight — Compact circuits, client-side proofs, testnet" + team/track card.

17.3 README checklist

- ☐ Pitch + hero screenshot (pygame + dashboard side-by-side) ☐ "The rigged dealer" before/after code snippet
- ☐ Architecture diagram (§2.2) ☐ Trust model table (§2.4) verbatim ☐ Claim-discipline note
- ☐ Quickstart: `venv` → `docker compose` → `compact compile` → `npm dev` → `python main.py` (tested on a clean machine)
- ☐ API reference summary (§9) ☐ Test instructions + tables (§15) ☐ Versions section (§4.5)
- ☐ Base-repo credit: "Engine derived from AI0702/Among-Us-clone (Unlicense); all Midnight integration built during the hackathon" ☐ Event rules on prior work checked & complied
- ☐ Known limitations (bridge custody, pickle, task-win attestation, bridge-crash) ☐ Roadmap (web client, player-held salts, mental-poker dealing, Lace) ☐ Video link · team · track

18. Troubleshooting Appendix

Symptom	Cause	Fix
<code>ModuleNotFoundError: asyncore</code>	Python $\geq$ 3.12	Use the 3.11 venv (§4.2) or pyasyncore shim; re-check <code>python --version</code> inside the venv
Proof generation hangs / connection refused :6300	Proof server container down or wrong network flag	<code>docker compose -f proof-server-local.yml logs</code> ; ensure <code>--network testnet</code> matches MN_NETWORK_ID
Contract deploy fails with balance error	No tDUST	Faucet the deployer address; confirm via <code>/health balanceTDust</code>
Generated bindings don't match spec names	Newer compiler renamed APIs	Expected — spec shapes are binding, identifiers aren't; align with generated <code>managed/</code> output
Version clash between @midnight-ntwrk packages	Mixed versions	Reset ALL to the example-bboard set (§4.5); delete <code>node_modules</code> + lockfile; reinstall
pygame clients desync after patches	Patch touched the 26-element pickle payload	P1–P5 must not change the payload array shape; hooks read state, never extend the wire format
Kill events double-post	<code>updateWorld()</code> processes <code>victim_id</code> on multiple frames	Debounce in P2: only fire hook on alive → dead transition; bridge 409 DEAD_TARGET is the backstop
Tx stuck "submitted" forever	Testnet congestion / dropped tx	Queue retry covers it; if persistent, restart bridge with MN_CONTRACT_ADDRESS set (rejoin, don't redeploy mid-match — v1 limitation, note in README)
Windows: venv/docker path issues	—	Prefer WSL2 for the whole stack; document whichever the team uses

19. Glossary

Compact	Midnight's smart-contract language; compiles circuits + TS bindings.
Circuit	Contract entry point; executes with ZK proof of correct execution over private witnesses.
Witness	Private input supplied by the calling driver (the bridge), never revealed on-chain; must always be validated against commitments in-circuit.
Ledger state	Public on-chain contract state (Maps, Sets, Counters...).
persistentCommit / persistentHash	Std-lib commitment/hash primitives used for role commitments and vote nullifiers.
disclose()	Compact's explicit marker for private-derived data entering public state — the selective-disclosure control point.
Proof server	Local Docker service that generates ZK proofs for circuit calls.
tDUST	Testnet token paying for transactions; from the faucet.
Nullifier	One-way tag proving "this credential acted this round" without revealing which credential — enforces one-vote-per-seat.
Anti-rig ordering	This project's core guarantee: seed + commitments reach the chain before any role reaches the game server's runtime state.

current Midnight documentation at build time.